

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



BÁO CÁO TUẦN 10

“ Xây dựng Website đặt sân thể thao tích hợp AI chatbot ”

Giảng viên hướng dẫn: Kim Ngọc Bách

Họ và tên sinh viên : Bùi Trung Đức

Mã sinh viên : B23DCCN166

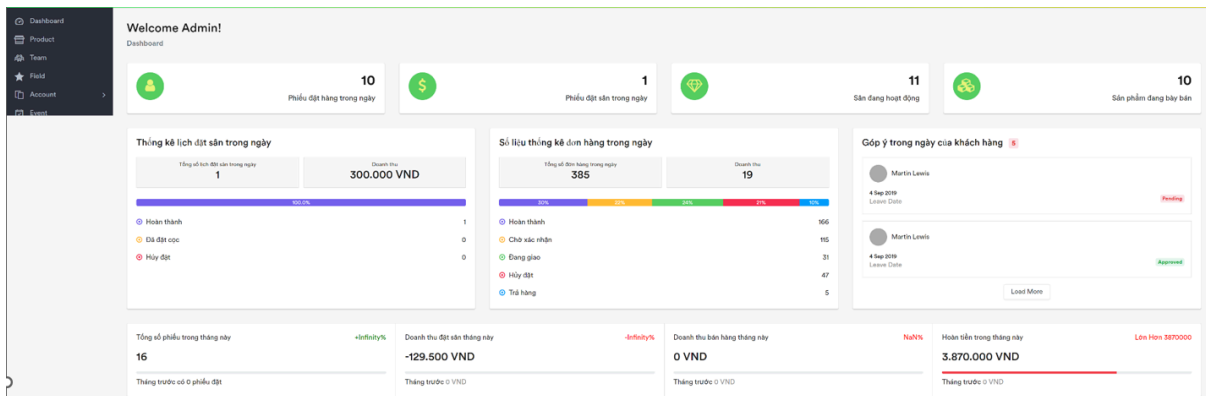
Lớp : D23CQCN12-B

Nhóm học phần : 11

Hà Nội, 5/2026

I. Công việc đã thực hiện được trong tuần

1. Hoàn thiện thêm giao diện trang Dashboard của Admin



- Người dùng có thể xem thêm các thông tin như tổng sản phẩm đang được đặt, sản phẩm đang hoạt động, số lịch trong ngày, doanh thu đặt sản của tháng, số tiền đã hoàn, góp ý trong ngày của khách hàng.

Top 5 khách hàng đặt sản nhiều nhất			
Name	Email	Status	Action
 Barry Cuda CEO	barrycuda@example.com	Active	...
 Tressa Wexler Manager	tressawexler@example.com	Inactive	...
 Ruby Bartlett CEO	rubybartlett@example.com	Inactive	...
 Misty Tison CEO	mistytison@example.com	Active	...
 Daniel Deacon CEO	danieldeacon@example.com	Inactive	...
View all clients			

- Ở cuối trang Dashboard, có thống kê những khách hàng đặt sản nhiều nhất của hệ thống

2. Frontend và Backend phần quản lý tài khoản.

a. Fronend

The Account management interface allows users to view and manage their account information. It includes a sidebar with 'Account' and 'Dashboard / Account' links. The main area features a search bar with filters for 'Full Name', 'Username', 'Trạng thái' (Status), and 'Quyền' (Permissions). Below the search bar, there is a table listing accounts with columns for STT (Serial Number), Fullname, Email, Phone, Address, Gender, and Action. The table shows 10 entries.

- Hiển thị danh sách các tài khoản
- Admin có thể tìm kiếm tài khoản, reset, thêm 1 tài khoản mới:

Add Employee



First Name *

Last Name

Username *

Email *

Password *

Address *

Phone *

Image*

Chọn tệp Chưa có tệp nào được chọn

Gender *

Nam ▼

Status *

Còn hoạt động ▼

Add Account

- Admin nhập các thông tin tài khoản của người dùng và có thể sửa và cấp quyền tài khoản thông qua nút “Edit”:

Role		
Admin	Staff	User
☒	☒	☒

[Edit Account](#)

b. Backend

1. *AccountRestController.java* (/rest/accounts/)

```

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidationException(MethodArgumentNotValidException ex) {
    return GlobalExceptionHandler.handleValidationException(ex);
}

@GetMapping("getAll")
public ResponseEntity<List<Users>> getAll(Model model){
    return ResponseEntity.ok(userDAO.findAll());
}

@GetMapping("get/{id}")
public ResponseEntity<Users> getOne(@PathVariable("id") String id) {
    if(!userDAO.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(userDAO.findById(id).get());
}

@PostMapping("create")
public ResponseEntity<Users> create(@Valid @RequestBody Users user) {
    if (user.getUsername() != null && userDAO.existsById(user.getUsername())) {
        return ResponseEntity.badRequest().build();
    }
    userDAO.save(user);
    return ResponseEntity.ok(user);
}

@PutMapping("update/{id}")
public ResponseEntity<Users> update(@PathVariable("id") String id, @Valid @RequestBody Users user) {
    if(!userDAO.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    userDAO.save(user);
    return ResponseEntity.ok(user);
}

```

```

@DeleteMapping("delete/{id}")
public ResponseEntity<Void> delete(@PathVariable("id") String id) {
    if(!userDAO.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    userDAO.deleteById(id);
    return ResponseEntity.ok().build();
}

@GetMapping("search")
public ResponseEntity<List<Users>> search(@RequestParam("user") String user,
    @RequestParam("keyword") String keyword,
    @RequestParam("status") Optional<Integer> status,
    @RequestParam("role") String role){
    return ResponseEntity.ok(userDAO.searchUserAdmin(user, keyword, status, role));
}
}

```

- REST API endpoints cho quản lý tài khoản admin
- Hỗ trợ: lấy danh sách (getAll), tìm kiếm (search), tạo (create), cập nhật (update), xóa (delete)
- Endpoint /search tìm kiếm tài khoản admin theo username, keyword, status, role

2. *AuthorizedRestController.java* (/rest/authorized/)

```

@PostMapping("create")
public ResponseEntity<Authorized> create(@RequestBody Authorized authorized) {
    if (authorized.getAuthorizedid() != null && authorizedDAO.existsById(authorized.getAuthorizedid()))
        return ResponseEntity.badRequest().build();
    authorizedDAO.save(authorized);
    return ResponseEntity.ok(authorized);
}

// lấy quyền của tài khoản
@GetMapping("getRole/{username}")
public ResponseEntity<List<Object[]>> getRole(@PathVariable("username") String username) {
    return ResponseEntity.ok(authorizedDAO.getRoleByUsername(username));
}

//xoa quyền
@DeleteMapping("/{authorizedid}")
public void delete(@PathVariable("authorizedid") Integer authorizedid) {
    authorizedDAO.deleteById(authorizedid);
}
}

```

- Quản lý quyền (roles) của tài khoản
- Endpoint để tạo phân quyền, lấy role của user, xóa role

3. *UserDAO.java* - DAO layer

- Query searchUserAdmin(): tìm kiếm tài khoản admin với bộ lọc
- Query findByUsername(): tìm thông tin user theo username

4. *UserService.java* / *ServiceImpl.java* - Service layer

- Thực hiện CRUD operations cho User

5. *HomeAdminController.java* - Controller

- Route trang admin (/sportify/admin → /admin/index.html)

II. Khó khăn trong tuần

- Vẫn còn một số lỗi liên quan đến Backend phần đặt sân như reload lại giao diện sau khi đặt, hiển thị thiếu trường thông tin bên Admin, ...
- Lỗi truy vấn bên *UserDAO.java* khiến lấy sai role thông tin tài khoản

III. Kế hoạch tuần tiếp theo

- Hoàn thiện lại toàn bộ giao diện
- Xử lý lại các lỗi liên quan đến Backend
- Tìm hiểu và tích hợp được AI chatbox,... vào hệ thống